

Minimum Replacements to Sort the Array

A Comprehensive, Step-by-Step Guide & Intuitive Mathematical Breakdown

1. The Core Paradigm Shift

LeetCode 2366. Minimum Replacements to Sort the Array is a problem where discovering the correct direction of traversal is the fundamental key to solving it efficiently.

The Crucial Insight: Right-to-Left Traversal

Most initial attempts fail by processing the array from **left to right**. Because splitting a number creates smaller numbers to its left, choices made on the left cascade unpredictably.

The trick: Process the array from **right to left** (backwards). The last element is already fixed and dictates the upper bound for all preceding elements!

2. Problem Definition & Operations

You are given an array of positive integers. You can replace any element with **two positive integers** whose sum equals the original number.

Original: 8 Split 1: 3 + 5 (1 operation) Split 2: 4 + 4 (1 operation) Goal: Make the final array non-decreasing with the minimum number of operations.

Each single replacement step increases the total count of numbers in the array by 1 and counts as **1 operation**.

3. Key Observation

Consider the sub-array `[16, 5]`:

- The last element (5) is fixed.
- We ask: "What is the largest value that can appear before 5?"
- Obviously, it cannot exceed 5 because $16 > 5$ violates the non-decreasing property.
- Therefore, we must split 16 into pieces that are each ≤ 5 .

4. Splitting & Operations Analysis

Example 1: Splitting 16 before 5

Can we split 16 into $8 + 8$?

```
16 5 → 8 8 5 (Invalid: 8 > 5)
```

To ensure every piece is ≤ 5 , we must divide 16 into at least 4 pieces:

```
16 → 4 4 4 4 Array: 4 4 4 4 5 (Sorted!)
```

Operation Count: Starting from 1 element, forming 4 pieces requires 3 replacements:

```
16 → 8 + 8 (1st replacement) → 4 + 4 + 8 (2nd replacement) → 4 + 4 + 4 + 4 (3rd replacement)
```

$$\text{operations} = \text{pieces} - 1$$

Example 2: Splitting 11 before 5

Need every part ≤ 5 .

```
11 → 5 + 5 + 1 (3 pieces) Operations = 3 - 1 = 2
```

5. Calculating the Minimum Pieces (Ceiling Logic)

Suppose *current* = 17 and our limit is *limit* = 5. We need all pieces to be ≤ 5 .

Why isn't simple integer division $17 / 5 = 3$ enough? Because 3 pieces of maximum size 5 sum to at most 15 ($5 + 5 + 5 = 15 < 17$). If we forced 3 pieces, one would be 7 ($5 + 5 + 7$), which violates $7 \leq 5$.

Therefore, we need the **ceiling** of division:

$$\text{pieces} = \lceil \text{current} / \text{limit} \rceil = \lceil 17 / 5 \rceil = 4$$

To perform ceiling division using purely integer arithmetic without floating-point inaccuracies:

$$\text{pieces} = (\text{current} + \text{limit} - 1) // \text{limit}$$

6. Updating the Limit for the Next Left Element

This is the most critical algorithmic step. When splitting 17 into 4 pieces, how should we split it?

Maximizing the Smallest Piece

If we split **17** as **[5, 5, 5, 2]**, the smallest piece is 2. That forces the element to its left to be ≤ 2 , requiring unnecessary extra splits!

To maximize the threshold for preceding elements, we must make the split as **balanced as possible**.

With **17** split into 4 balanced pieces, the elements are **[4, 4, 4, 5]**. The smallest piece in a balanced distribution is given by integer division:

$$\text{new_limit} = \lfloor \text{current} / \text{pieces} \rfloor = 17 // 4 = 4$$

Thus, the next number on the left must be ≤ 4 .

7. Complete Step-by-Step Algorithm Walkthrough

Let us trace the algorithm on the array: `nums = [12, 9, 7, 6, 17, 19, 21]`

Index	Element ()	Current	Condition / Action	Pieces	Ops Added	New
6	21	∞	Initial fixed tail	1	0	21
5	19	21	$19 \leq 21$ (Valid)	1	0	19
4	17	19	$17 \leq 19$ (Valid)	1	0	17
3	6	17	$6 \leq 17$ (Valid)	1	0	6
2	7	6	$7 > 6$ (Requires split)	$\lfloor 7/6 \rfloor = 2$	+1	$\lfloor 7/2 \rfloor = 3$
1	9	3	$9 > 3$ (Requires split)	$\lfloor 9/3 \rfloor = 3$	+2	$\lfloor 9/3 \rfloor = 3$
0	12	3	$12 > 3$ (Requires split)	$\lfloor 12/3 \rfloor = 4$	+3	$\lfloor 12/4 \rfloor = 3$

Total Operations Calculated: $0 + 0 + 0 + 0 + 1 + 2 + 3 = 6$ operations.

Algorithm Summary & Formula Reference

- **Traversal Direction:** Right-to-Left (from $n - 2$ down to 0).
- **Initial Limit:** Set $\text{limit} = \text{nums}[n - 1]$.
- **Number of Pieces:** $\text{pieces} = (\text{nums}[i] + \text{limit} - 1) // \text{limit}$
- **Operations Count:** $\text{operations} += \text{pieces} - 1$
- **New Limit Update:** $\text{limit} = \text{nums}[i] // \text{pieces}$